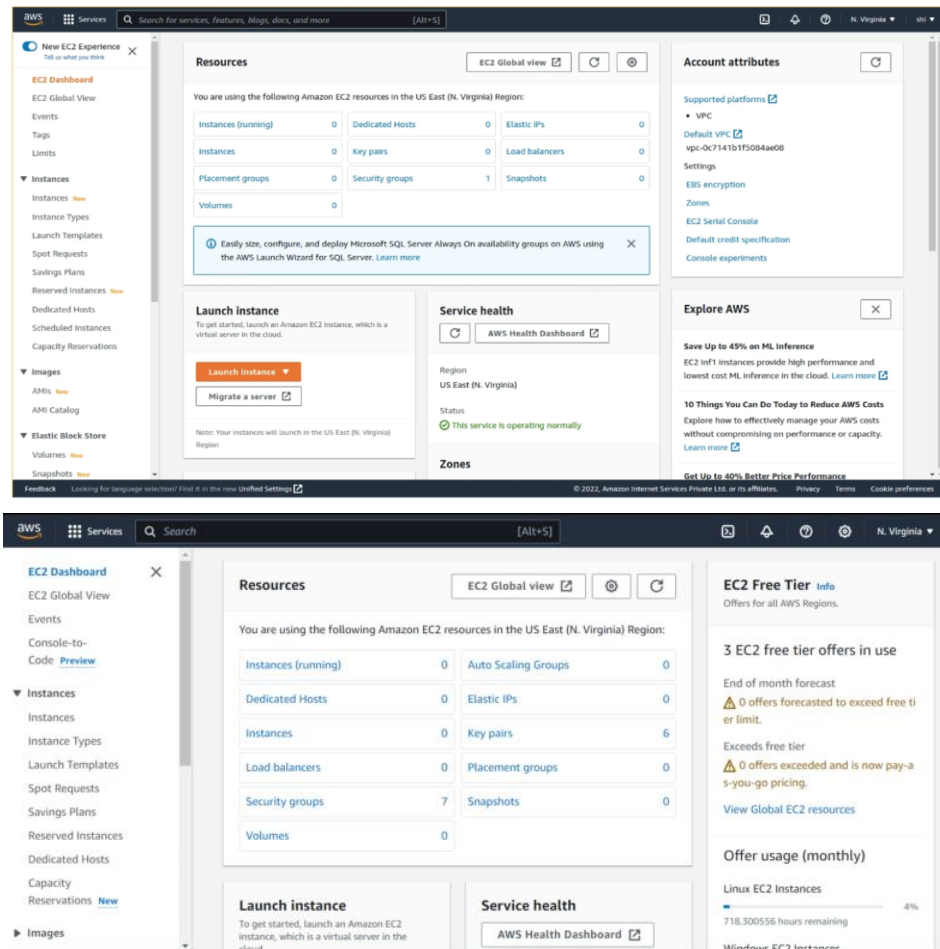
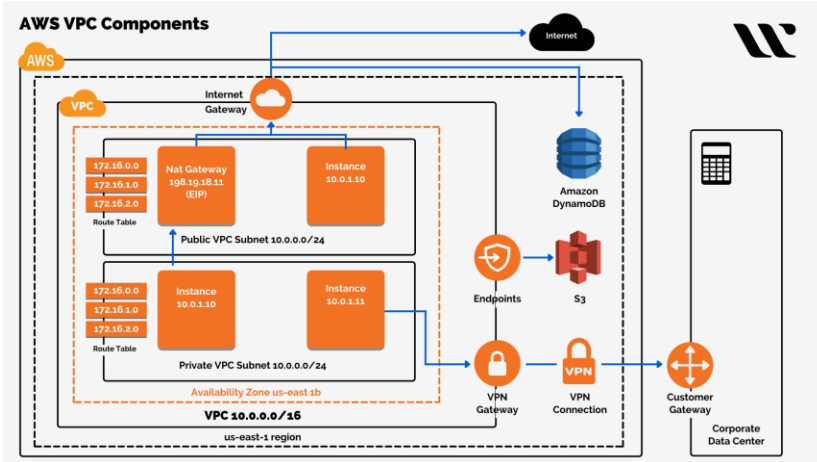
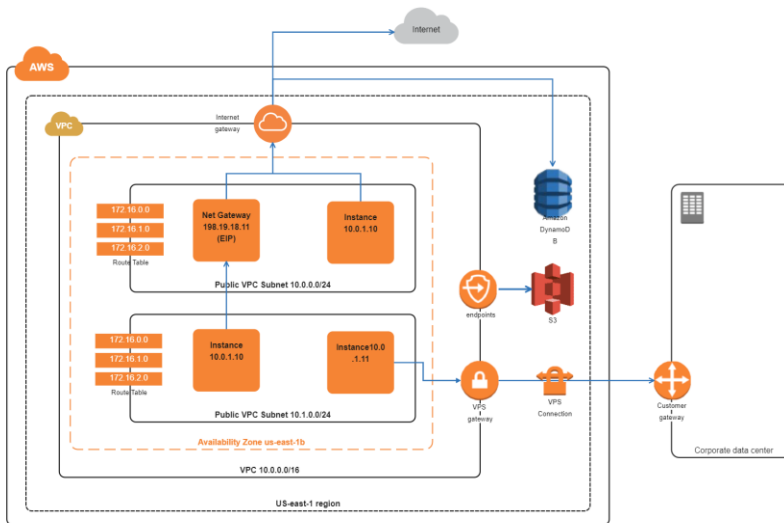


Launching an EC2 Server for a Shopping Web App

A practical, step-by-step guide from the perspective of an AWS Cloud Engineer.



AWS VPC Components



1. Choose the Correct Region

Pick the AWS Region closest to your customers to reduce latency. For a shopping app serving U.S. users, **us-east-1** or **us-east-2** is typical.

- This ensures faster page loads and smoother checkout flows.
- It also helps keep costs predictable and aligns with compliance expectations.

2. Open EC2 and Start the Launch Wizard

Navigate to **EC2 → Launch Instance**. This is where you define the compute environment for your web app.

Use a clear name **such as** **shopping-web-app-prod-01**.

3. Select the AMI (Operating System)

For web apps, I typically choose:

- **Amazon Linux 2** — stable, lightweight, and optimized for AWS
- **Ubuntu LTS** — great if your team prefers Ubuntu tooling

Your AMI should support your stack (Node.js, Python, Java, PHP, etc.).

4. Choose the Instance Type

For a shopping web app, traffic patterns matter:

- **t3.medium** — good baseline for moderate traffic
- **t3.large** — if you expect heavier browsing or flash-sale spikes
- **m5 family** — for more predictable performance

You want enough CPU/RAM to handle product browsing, cart updates, and checkout API calls.

5. Configure Key Pair

Create or select a key pair for SSH access.

- Store the `.pem` file securely.
- Limit access to engineering team members only.

6. Configure Networking (Critical for Web Apps)

This is where shopping apps differ from generic EC2 setups.

VPC & Subnet

- Place the instance in a **public subnet** if it **serves** traffic directly.
- Place it in a **private subnet** if it sits behind a load balancer (recommended for production).

Security Group

Add inbound rules:

- **HTTP (80)** — for web traffic
- **HTTPS (443)** — for secure checkout
- **SSH (22)** — restricted to your office/VPN IP range

This ensures **that** your app is reachable but still protected.

Create Security Group

Security group name ⓘ

Web Server Security Group

Description ⓘ

Security for production web server.

VPC ⓘ

vpc-e68d9c81 | DefaultVPC (default) ▾

Security group rules:

Inbound

Outbound

Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ		Description ⓘ
SSH ▾	TCP	22	Anywhere ▾	0.0.0.0/0, ::/0	Admin access.
HTTP ▾	TCP	80	Anywhere ▾	0.0.0.0/0, ::/0	Web traffic.
HTTPS ▾	TCP	443	Custom ▾	0.0.0.0/0, ::/0	Secure web traffic.

Add Rule

<

>

Cancel

Create

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 194.44.90.91/32, My IP for configuration) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Name Info	Description - optional Info
My IP	Add CIDR, prefix list or security 194.44.90.91/32	My IP for configuration

▼ Security group rule 2 (TCP, 22, 52.3.83.134/32, data.world inbound connection) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security 52.3.83.134/32	data.world inbound connection

▼ Security group rule 3 (TCP, 22, 52.205.195.10/32, data.world inbound connection) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security 52.205.195.10/32	data.world inbound connection

▼ Security group rule 4 (TCP, 22, 52.205.207.86/32, data.world inbound connection) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security 52.205.207.86/32	data.world inbound connection

Add security group rule

7. Configure Storage

Your shopping app will generate logs, images, and cached data.

- Use **gp3** EBS volumes for performance.
- Start with **20–30 GB** for the root volume.
- Add a second volume if your app stores product images locally (though S3 is preferred).

8. Add Tags for Governance

Tags matter for cost tracking and automation:

- *Name: shopping-web-app-prod-01*
- *Environment: production*
- *Application: shopping-web-app*
- *Owner: engineering*

These help with monitoring, billing, and scaling.

9. Review and Launch

Double-check:

- *AMI*
- *Instance type*
- *Security group*
- *Storage*
- *Tags*

Then click **Launch Instance**.

10. Connect and Configure the Server

SSH into the instance and install your stack:

- *Web server (Nginx or Apache)*
- *Runtime (Node.js, Python, PHP, Java)*
- *App code deployment*
- *Environment variables (DB credentials, API keys)*
- *SSL certificates (via ACM if using a load balancer)*

11. Integrate with Other AWS Services

A shopping app typically uses:

- **RDS or DynamoDB** for product/catalog data
- **S3** for product images

- **CloudFront** for CDN caching
- **ALB** for load balancing
- **CloudWatch** for monitoring
- **IAM roles** for secure access to AWS resources

These services ensure scalability during peak shopping periods.

12. Test the Deployment

Verify:

- *Homepage loads*
- *Product pages respond quickly*
- *Cart and checkout APIs work*
- *HTTPS is enforced*
- *Logs are flowing to CloudWatch*

13. Prepare for Scaling

Shopping apps often need elasticity:

- *Configure **Auto Scaling Groups***
- *Add **Load Balancers***
- *Enable **CloudWatch alarms** for CPU, latency, and error rates*

